FEAT-ZD-48600 · DE-06 Data Mapping · Updated data mappings and transformation steps

Data Engineering agent · item feat.mappings · source hobasa-data-engineering-agent/outputs/FEAT-ZD-48600/feat.mappings.md · approved by Data engineer sim.data_engineer

feat.mappings: FEAT-ZD-48600

Updated data mappings and transformation steps for the Data Engineering lane of FEAT-ZD-48600 ("Show which overdue invoices belong to vendors that usually pay late"). This is DE-06, Data Mapping, attempt 1. The workbook action is "Update the data mapping based on the data model and fetch the latest data mappings based on the customer input". There is no gate: automated checks, the independent judge and sampled human evaluation review this output (record:brief).

Field	Value	Source
Trace ID	FEAT-ZD-48600	record:brief
Step	DE-06 Data Mapping, attempt 1	record:brief
Inputs	feat.data_model_change (379f011, from DE-05), org.customer_mappings (a6b3e2a, feature org-override)	record:brief; feat.data_model_change; org.customer_mappings
Code read	ap_pipeline source, tests and `schedule.json`, as given in the step's tool view	tool:repo
Customer mapping version applied	Vendor feed **v3**: `invoice_id, vendor, due_date, amount, paid, paid_date`; `paid_date` optional, ISO date, empty when unpaid	org.customer_mappings
Result	**The customer contract adds one mapping,** `paid_date -> Invoice.paid_date`**; the five existing column mappings are unchanged. The mapping agrees with the Data Model change field by field. The schedule stays daily 02:00 Asia/Kolkata, matching** `schedule.json`**.** No new source, no new target entity and no storage change	org.customer_mappings; feat.data_model_change (2.1, 2.2, 3); tool:repo (`schedule.json`, `transform.py`)

1. Latest customer mapping contract (fetched)

Item	Customer contract (org.customer_mappings)	Data Model change (feat.data_model_change)	Agreement	Source
Feed version	v3	v2 today, v3 adds the optional `paid_date` column	Agrees	org.customer_mappings; feat.data_model_change (2.1)
Columns	`invoice_id, vendor, due_date, amount, paid, paid_date`	Same five existing columns plus `paid_date`	Agrees	org.customer_mappings; feat.data_model_change (2.1)
`paid_date` spelling	`paid_date`	Assumed `paid_date` (open question DMQ-B)	Agrees; the customer contract confirms the spelling (DMQ-B can close, see section 6)	org.customer_mappings; feat.data_model_change (6)
`paid_date` format	Optional, ISO date, empty when unpaid	ISO date string or blank; blank or missing gives `None`	Agrees	org.customer_mappings; feat.data_model_change (2.1, 2.2)

Item	Customer contract (org.customer_mappings)	Data Model change (feat.data_model_change)	Agreement	Source
Target	`paid_date -> Invoice.paid_date`	Invoice `paid_date`: date, nullable	Agrees	org.customer_mappings; feat.data_model_change (2.2)
Schedule	Daily 02:00 Asia/Kolkata (unchanged)	Not in scope of the model	Matches `schedule.json` (`"frequency": "daily"`, `"time": "02:00"`, `"timezone": "Asia/Kolkata"`)	org.customer_mappings; tool:repo (`schedule.json`)

2. Source-to-target mapping: feed row → Invoice

Source: vendor invoice feed row (CSV with a header, parsed by `ingest.parse_feed` into a `dict` keyed by header). Target: Invoice record (plain `dict`) returned by `transform.to_model` (tool:repo `ingest.py`, `transform.py`; feat.data_model_change 2.2).

Source column	Target field	Target type	Nullable	Transformation rule	Change	Source
`invoice_id`	`id`	str	No	`strip()`	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`vendor`	`vendor`	str	No	`strip()`	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`due_date`	`due_date`	date	No	`date.fromisoformat(...)`; no `strip()` added	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`amount`	`amount`	Decimal	No	`Decimal(...)`; never float	None	tool:repo (`transform.py`); feat.data_model_change (2.2); record:memory (D-2026-044, L-011)
`paid`	`paid`	bool	No	`strip().lower() == "true"`; missing column → `false`	None	tool:repo (`transform.py`); feat.data_model_change (2.2)
`paid_date`	`paid_date`	date	**Yes (`None`)**	Missing key (feed v2), `None` (short row) or blank after `strip()` → `None`; otherwise `date.fromisoformat` of the stripped value; a non-blank invalid value rejects the row with reason `paid_date`	**New mapping**	org.customer_mappings; feat.data_model_change (2.1, 2.2, 2.3)

3. Transformation steps (per feed row, in this order)

The order keeps today's behaviour for the five existing fields and adds `paid_date` last, so an existing reject reason wins over `paid_date` (feat.data_model_change 2.2, DD-6).

1 **Parse** the CSV text into header-keyed rows; unchanged, no code change for v3 because `csv.DictReader` keys rows by header (tool:repo `ingest.py`; feat.data_model_change 2.1).

2 **Map the existing fields** `id`, `vendor`, `due_date`, `amount`, `paid` exactly as today. A missing or invalid field rejects the row with the exception type name as `reason` (`KeyError`, `ValueError`, `InvalidOperation`, `AttributeError`) (tool:repo `transform.py`; feat.data_model_change 2.3).

3 **Map** `paid_date` only when step 2 succeeded: take `row.get("paid_date")`; if it is `None` or blank after `strip()`, set `paid_date = None`; otherwise parse the stripped value as an ISO date (org.customer_mappings; feat.data_model_change 2.2).

4 **Reject on invalid** `paid_date`: if the parse in step 3 fails, append `{"row": row, "reason": "paid_date"}` (constant `PAID_DATE_REASON`) to the rejected list and do not load the record (feat.data_model_change 2.3).

5 **Emit the record** with exactly the keys `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`; `paid_date` is present on every accepted record, including feed v2 rows (feat.data_model_change 2.2, 4).

6 **Keep record-level rules unchanged**: a paid record with `paid_date = None` is kept; an unpaid record with a valid `paid_date` is kept (interim, DD-7); a `paid_date` earlier than `due_date` is kept (feat.data_model_change 2.2).

7 **Data-quality report**: unchanged code; `reject_reasons` is the sorted set of rejected `reason` values, so `paid_date` appears on its own (tool:repo `dq.py`; feat.data_model_change 2.3).

8 **Derive the Delay history** with the new pure function `delays.delays_by_vendor(records)` (section 4).

4. Derived mapping: Invoice → Delay history

Source (Invoice)	Target	Rule	Source
`vendor`	Key of `Dict[str, List[int]]`	Vendors in first-seen order; a vendor with no qualifying record has no key	feat.data_model_change (2.4)
`paid_date`, `due_date`	`delay_days` list value	`(paid_date - due_date).days`, calendar days, int, may be negative; values in feed order	feat.data_model_change (2.4); record:memory (D-2026-031)
`paid`, `paid_date`	Filter	Only records where `paid` is truthy **and** `paid_date` is not `None`	feat.data_model_change (2.4)
—	Persistence	None; in memory only; the record list is not modified	feat.data_model_change (2.4)
—	Consumer	ap_forecast `vendor_risk(history)` takes one vendor's list	feat.data_model_change (2.4, 2.5)

5. Schedule mapping

Attribute	Customer contract	Pipeline config today	Change	Source
Frequency	Daily	`"frequency": "daily"`	None	org.customer_mappings; tool:repo (`schedule.json`)
Time	02:00	`"time": "02:00"`	None	org.customer_mappings; tool:repo (`schedule.json`)
Timezone	Asia/Kolkata	`"timezone": "Asia/Kolkata"`	None	org.customer_mappings; tool:repo (`schedule.json`)

This step changes no schedule; it only confirms that the customer frequency and the config agree (org.customer_mappings; tool:repo `schedule.json`).

6. Mapping validation rules (for the coding and unit-test steps)

Case (synthetic rows only)	Expected	Source
v2 header (no `paid_date` column), valid row	Loaded; `paid_date = None`	feat.data_model_change (3, 4)
v3 row, `paid_date` blank (unpaid, as the customer sends it)	Loaded; `paid_date = None`	org.customer_mappings; feat.data_model_change (4)
v3 row, `paid_date` valid ISO date	Loaded; `paid_date` is a `date`	org.customer_mappings; feat.data_model_change (4)
v3 row, `paid_date` non-blank and invalid (for example `"2026-13-01"`, `"not-a-date"`)	Rejected, reason `paid_date`, listed in `reject_reasons`	feat.data_model_change (4)
Bad `due_date` and bad `paid_date` together	Rejected, reason `ValueError`	feat.data_model_change (4)
Every accepted record	Keys exactly `id`, `vendor`, `due_date`, `amount`, `paid`, `paid_date`; `amount` is `Decimal`	feat.data_model_change (4); record:memory (D-2026-044)

7. Code in this step

This step updates the mapping document only and changes no code. The tool view shows `transform.py` without `paid_date` and no `delays.py` module (tool:repo). The mapping is implemented by TC-1 and TC-2 in the later Data Engineering coding step (feat.data_model_change 5).

8. Open questions (non-blocking)

ID	Question	Interim answer	Owner (proposed)
MQ-A	DMQ-B (is the v3 column spelled exactly `paid_date`?) is answered by the customer contract, which lists `paid_date`. Can DMQ-B be closed?	Yes; this mapping uses `paid_date`	Data engineering lead
MQ-B	The customer contract says `paid_date` is "empty when unpaid", while the model keeps an unpaid record that carries a valid `paid_date` (DD-7, DMQ-C, interim). Should such a row instead be rejected or have `paid_date` cleared?	Keep it as the model says; the Delay history ignores it because it filters on `paid`	Product / Architect; feed owner
MQ-C	The customer contract does not version its mapping beyond "v3" or state an effective date for v3. Does the pipeline need to accept v2 and v3 side by side for a period?	Yes; the mapping already accepts both (missing column → `None`)	Data engineering lead / feed owner

9. Validation check

Check	Result	Source
Mapping updated from the data model	Pass: every Invoice field in feat.data_model_change 2.2 has one source column (section 2); Delay history mapped (section 4)	feat.data_model_change

Check	Result	Source
Latest customer mapping applied	Pass: v3 columns and `paid_date -> Invoice.paid_date` (sections 1 and 2); schedule confirmed (section 5)	org.customer_mappings
Conflict with the Data Model change or the customer contract	None; one wording tension raised as MQ-B	org.customer_mappings; feat.data_model_change
Production or source-system data read	None; only the inputs, the tool view and synthetic rows	record:brief
Blocking questions	None	—